

TRPO 实验

高进轩

2026 年 5 月 19 日

1 引言

本实验使用 TRPO 算法,对 **Humanoid-v5**, **Ant-v5**, **HalfCheetah-v5**, **Hopper-v5**, **Walker2d-v5** 环境进行训练和测试。先对五个环境全部进行短期测试,对不同训练方式进行对比,一共 20 组实验,大多数策略都在单调提升。接着选取 **HalfCheetah-v5**, **Hopper-v5**, **Walker2d-v5** 进行长期测试,其中 **HalfCheetah-v5**, **Hopper-v5** 中 reward 明显提升且达到阶段性收敛,**Walker2d-v5** 仍然波动明显。为此在长期训练的基础上补充了额外训练,效果仍然不佳,又更换超参数重新实验,最后看到 reward 曲线表现良好且阶段性收敛。在完成实验后,又对理论性和实验中的一些问题做出了回答。

2 代码补全与理论问题

该项目的代码实现和理论依据参考了 Open AI 发表的 Trust Region Policy Optimization(<https://spinningup.openai.com/en/latest/algorithms/trpo.html>) 以及 John Schulman 发表的 Trust Region Policy(<https://arxiv.org/abs/1502.05477>)。

2.1 collect_trajectories 实现部分

1. collect_trajectories 与普通 rollout 采样方式相比,数据收集的停止条件分别是什么?

答:前者须搜集到指定轨迹条数时停止,每一条轨迹要么是走到终点要么达到最大步数。而后者是智能体交互一定次数后就停止,此时可能停在一条轨迹的一半位置,但是到了规定步数就停。

2. 在 trajectory 采样模式下,每一条轨迹需要保存哪些信息? 这些信息分别在后续 TRPO 更新中起什么作用?,当环境返回终止信号时,轨迹又会如何结束? 未达到最大长度的部分又应如何处理?

答:每一条轨迹保存 states,actions,rewards,dones,log_probs,masks。

states,actions,rewards 的作用无需多言,用来更新各种参数。dones 记录轨迹是否终止。log_probs 用来计算 $\log \pi(a|s)$ 从而计算 g,由于替代优势函数的梯度在 θ_k 处为

$$g = \mathbb{E}_{s \sim \rho_{\theta_k}, a \sim \pi_{\theta_k}} \left[\left. \nabla_{\theta} \log \pi_{\theta}(a | s) \right|_{\theta=\theta_k} A^{\pi_{\theta_k}}(s, a) \right]$$

dones 用于记录一条轨迹是否终止。masks 用于记录该处数据是真实的还是填充的,因为不是每条轨迹都能到达最大步数。

当 $done = true$ 时,这条轨迹就终止并存储数据,没达到最大步数的其余空间全部用 0 填充并标记 $mask = 0$ 。

3. TrajectoryRollout 中的 masks 变量有什么作用？如果不使用 masks，计算 return 或 advantage 时可能出现什么问题？

答：mask 的作用见上一问。很显然若没有 mask 变量，因轨迹步数未达到最大步数而填充的一堆 0 会被错误地认为是真实数据，从而使计算的 return 和估计的 advantage 都被拉低造成很大的 bias。

4. collect_trajectories 在收集完一批轨迹后为什么需要重新初始化环境状态？

答：在每轮采样完后，最后一条轨迹可能并没有走到环境的终止位置，二是而是达到了最大步数，所以需要重置以免造成混乱。重新初始化保证新一轮采样都是从初始位置开始，且新一轮的所有数据都是从头依据新策略产生，防止上一轮旧策略的交互产生的影响符合 TRPO 的 on-policy 特性。

2.2 __update_policy 实现部分

Theorem 1. Let $\alpha = D_{TV}^{\max}(\pi_{old}, \pi_{new})$. Then the following bound holds:

$$\eta(\pi_{new}) \geq L_{\pi_{old}}(\pi_{new}) - \frac{4\epsilon\gamma}{(1-\gamma)^2}\alpha^2$$

(1)

where $\epsilon = \max_{s,a} |A_{\pi}(s,a)|$

1. 解释该公式中每一项的实际意义，这个公式在什么情况下可以保证策略的单调改进？由此直观解释 TRPO 算法的设计思路（优化目标和约束的设计？先概述你的大致想法，具体细节在后续提问中体现）。

答： $\eta(\pi_{new})$ 表示的是新策略的真实期望价值。 $L_{\pi_{old}}(\pi_{new})$ 表示替代函数，由于新策略真实的期望函数需要基于新策略的分布来求期望，可是我们站在旧策略上得不到新策略下采样出来的数据，因此运用了近似的手法，用旧策略的分布构造了一个对新策略的估计函数。最后一个复杂的项就是对真实值和估计值之间的误差分析，这里使用了 TV 散度来估计两个策略分布之间的“距离”，只要不太大，那么估计值就可以近似成真实值，从而保证了单调改进。因此 TRPO 的简要思路可以说是：通过人为计算每一次迭代的步长来确保策略总是在单调变好的，而在具体实现时需要用到近似方法，由此需要一个“距离”（TV 或 KL 散度）的约束来确保近似值不会差很远，因此实际实现目标就是最大化替代函数的同时满足一定的散度约束条件。

2. 和文章 Approximately optimal approximate reinforcement learning 中的 Theorem 4.1（即 TRPO 论文中的公式 (6)）作对比，TRPO 作者在分析上的处理技术有什么具体的不同，使得 TRPO 可以推广到所有一般的随机策略类而不是给定的混合策略类？

答：主要的不同在于：

$$\eta(\pi_{new}) \geq L_{\pi_{old}}(\pi_{new}) - \frac{2\epsilon\gamma}{(1-\gamma)^2}\alpha^2$$

where $\epsilon = \max_s |\mathbb{E}_{a \sim \pi'(a|s)} [A_{\pi}(s,a)]|$.

中的 α 是由

$$\pi_{new}(a|s) = (1-\alpha)\pi_{old}(a|s) + \alpha\pi'(a|s).$$

所定义的。这里只考虑到了混合策略。而 TRPO 中 α 的选取使用了 KL 散度或 TV 散度来刻画策略之间的“距离”，推广到了任意类型的策略，因此更适用于深度强化学习。

3. TRPO 中策略更新的 surrogate objective $L_{\pi_{old}}(\pi_{new})$ 一项在实际算法中是如何设计的？

答：主要使用了泰勒展开，展开到一阶梯度项，原函数在 π_{old} 处的值恰好为 0，因此 $L_{\pi_{old}}(\pi_{new})$ 可以近似为 $g^\top(\theta' - \theta_k)$ ，其中 θ' 和 θ_k 分别代表新旧策略参数，而

$$g = \mathbb{E}_{s \sim \rho_{\theta_k}, a \sim \pi_{\theta_k}} \left[\nabla_{\theta} \log \pi_{\theta}(a | s) \Big|_{\theta=\theta_k} A^{\pi_{\theta_k}}(s, a) \right]$$

g 中的优势函数通过广义优势估计来估计。

4. 为什么 TRPO 需要约束新旧策略之间的 KL divergence 而不是定理中的 TV 距离？在实际算法中我们采用 KL 散度的二阶近似 $D_{KL}(\pi_{\theta_{old}} \parallel \pi_{\theta}) \sim \frac{1}{2} \Delta \theta^T (\nabla_{\theta}^2 D_{KL}) \Delta \theta$ ，延续该思路解释为什么在 Algorithm 1 中我们会采用 line search 搜索参数 (Algorithm 1 line 8&9)？

答：KL 散度相比 TV 散度在工程实现上的优势在于，它可导，计算方便，并且可以通过二阶泰勒展开的方式近似计算，大大降低计算成本。由上一问知我们的优化目标——替代函数也用一阶泰勒展开近似计算，约束条件也用二阶泰勒展开近似计算。都是近似计算，所以需要线性搜索的方法来保证最后得到的新策略即单调提升了也满足了约束条件，简单说就是在多步近似后的一个安全检查手段，确保“近似”没有跑的太偏。

3 实验设计

此次实验共三个阶段。首先对五个环境全部进行短期四组对比实验 (trajectories/rollout, advan_norm=true/false)。短期结果表明策略已经在多数环境中单调提升，并且在不同采样方式中产生对比。其次选取 HalfCheetah-v5、Hopper-v5 和 Walker2d-v5 进行长期训练，长期结果显示，Hopper-v5 的训练曲线最稳定，基本达到阶段性收敛；HalfCheetah-v5 也表现出较稳定的提升趋势，接近阶段性收敛；而 Walker2d-v5 虽然 reward 有所提升，但后期仍存在明显波动，尚未达到稳定收敛。再对 Walker2d-v5 补充了额外训练，发现还是波动很大，推测统一设置的超参数不适合于该环境，因此又重新设置了参数，获得了远超之前的 reward 曲线和阶段性收敛。

3.1 五个环境短期训练对比

短期实验在五个 MuJoCo 环境上全部进行四组对比：rollout/trajectory 两种采样方式，以及 advan_norm=true/false 两种 advantage normalization 设置。短期训练的目的不是追求完全收敛，而是观察不同环境的初始学习难度、不同采样方式的早期表现，以及 advantage normalization 是否会带来稳定提升。下列曲线均为平均测试回报 (mean test reward) 随交互步数变化的对比图。

3.1.1 Humanoid-v5

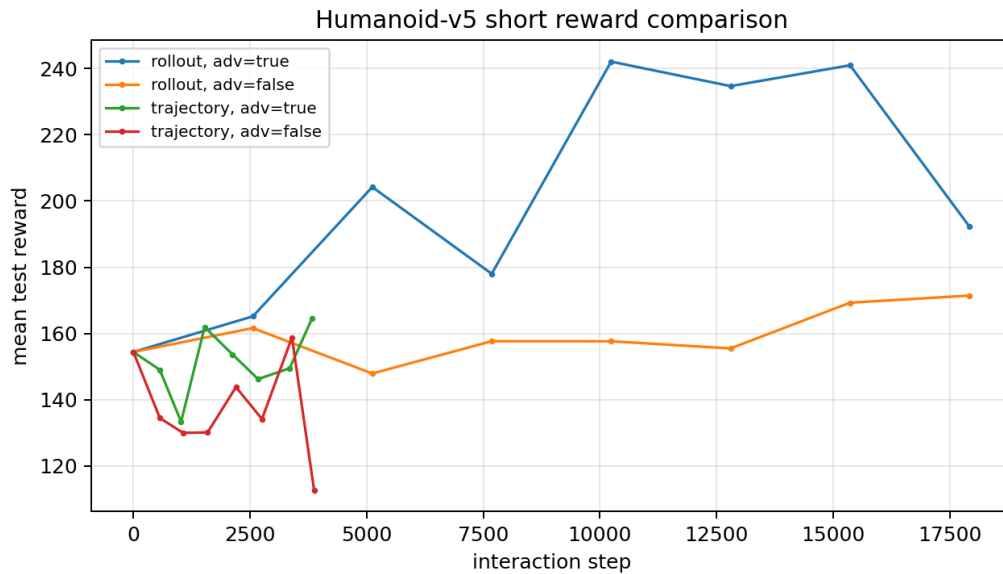


图 1: Humanoid-v5 短期训练 reward 曲线对比

Humanoid-v5 是五个环境中维度最高、控制最复杂的任务，因此短期训练中四组曲线波动都比较明显。结果上，rollout + advan_norm=true 的最好值较高，短期内曾达到约 242，但最后回落到约 192；trajectory 两组由于有效交互步数较少，提升更不稳定。整体来看，Humanoid-v5 在短期内能看到一定学习趋势，但远不能认为已经收敛。

3.1.2 Ant-v5

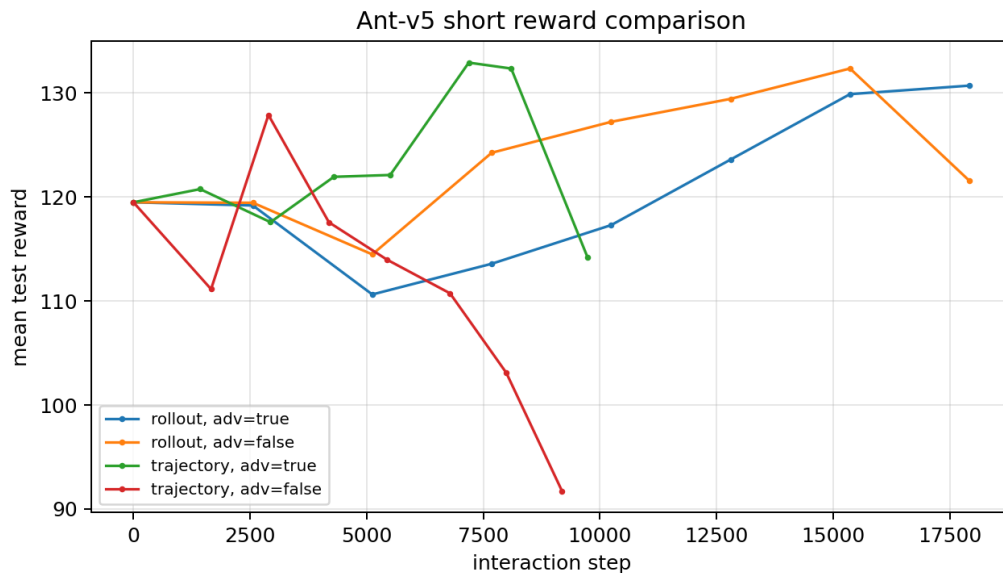


图 2: Ant-v5 短期训练 reward 曲线对比

Ant-v5 的短期曲线整体比 Humanoid-v5 稳定一些，rollout 两组最终 reward 分别约为 121.6 和 130.7，表现优于两组 trajectory。其中 trajectory 模式在短期内存在后期回落，说明在

该环境中按完整轨迹采样会减少每次更新的有效样本数，短期样本利用效率不如固定步数的 rollout。

3.1.3 HalfCheetah-v5

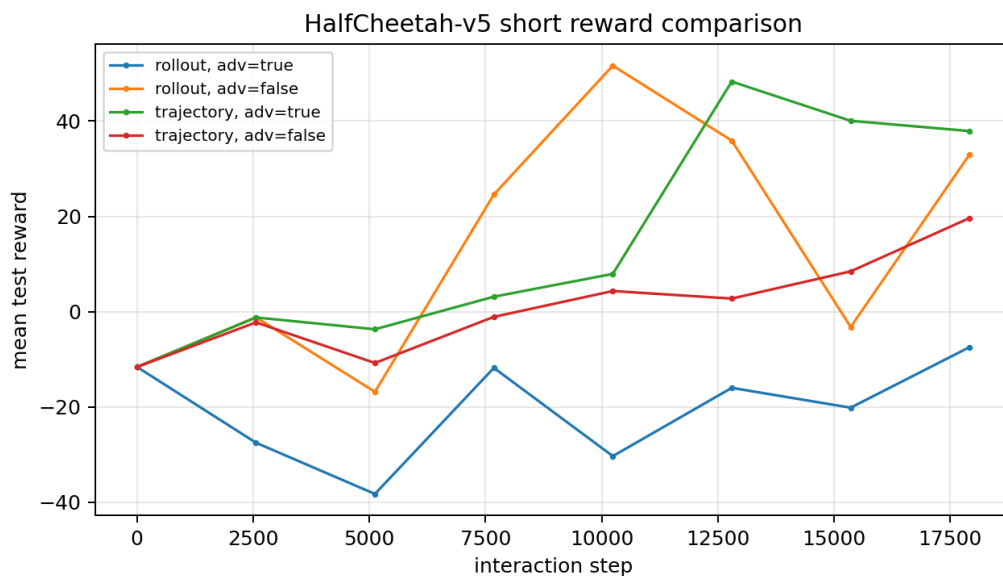


图 3: HalfCheetah-v5 短期训练 reward 曲线对比

HalfCheetah-v5 在短期训练中已经能看出不同配置之间的差异，但整体 reward 还处在较低区间。`trajectory + advan_norm=true` 和 `rollout + advan_norm=false` 出现过相对较好的结果，而 `rollout + advan_norm=true` 表现较弱。该环境短期内尚未充分发挥出可训练性，需要更长训练才能体现稳定趋势。

3.1.4 Hopper-v5

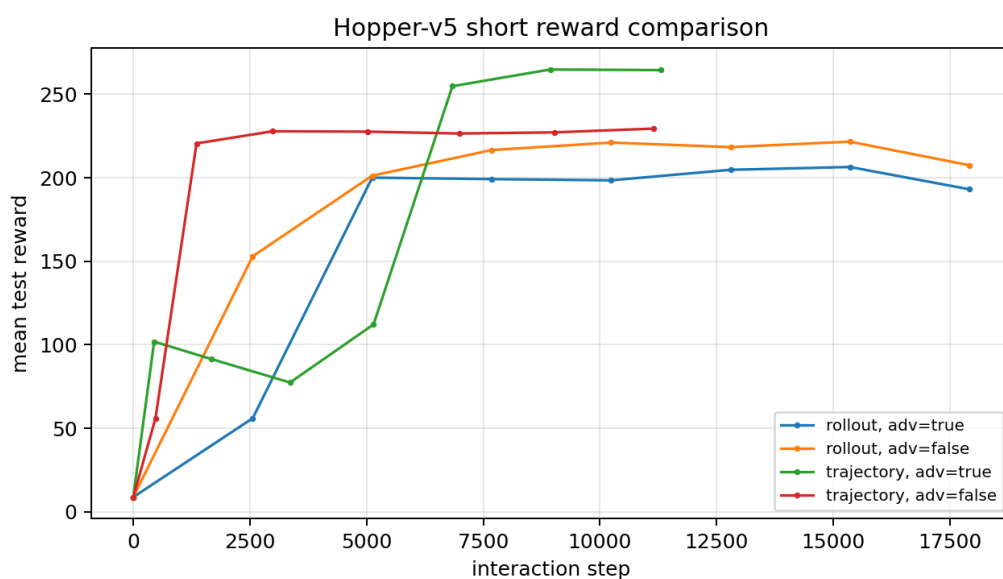


图 4: Hopper-v5 短期训练 reward 曲线对比

Hopper-v5 是短期实验中表现最明显的环境之一。四组配置都能较快获得较高 reward，其中 `trajectory + advan_norm=true` 最终约为 264.4，`trajectory + advan_norm=false` 约为 229.4，两个 rollout 配置也稳定在 190–207 左右。这说明 Hopper-v5 的训练难度相对较低，在较少交互步数下就能体现出明显的策略改进。

3.1.5 Walker2d-v5

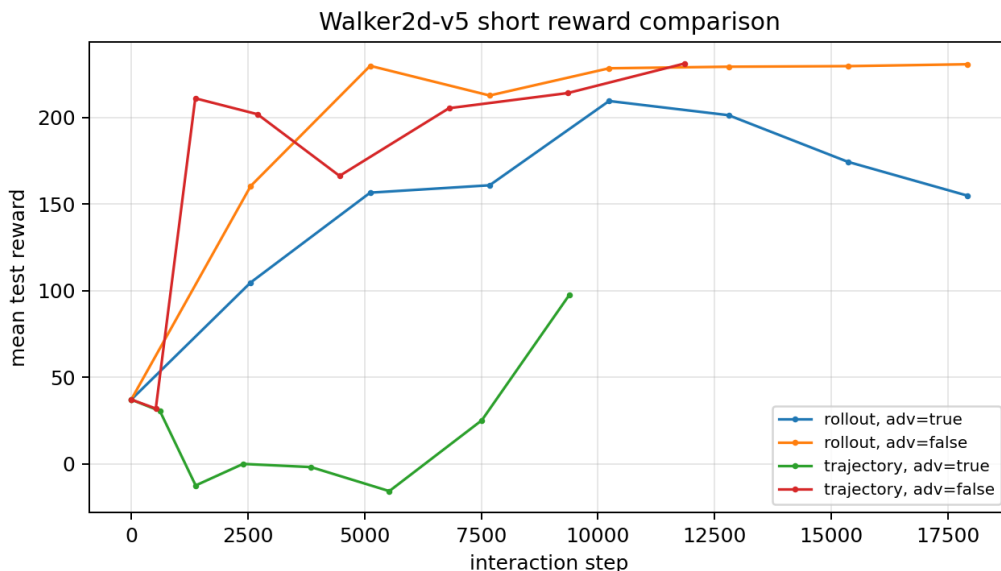


图 5: Walker2d-v5 短期训练 reward 曲线对比

Walker2d-v5 在短期实验中也能较快提升，其中 `rollout + advan_norm=false` 和 `trajectory + advan_norm=false` 的最终 reward 都在 230 左右，明显优于 `advan_norm=true` 的两组。短期结果提示该环境中 advantage normalization 未必总是有利，尤其在当前超参数下，未归一化 advantage 的两组表现更好。

3.2 三个环境长期训练对比

长期实验选择 HalfCheetah-v5、Hopper-v5 和 Walker2d-v5 三个环境，每个环境仍然保留四组配置对比。长期训练的目的在于观察短期结论是否能够延续，以及曲线是否进入相对稳定的平台区间。下列曲线同样为平均测试回报随交互步数变化的对比图。

3.2.1 HalfCheetah-v5

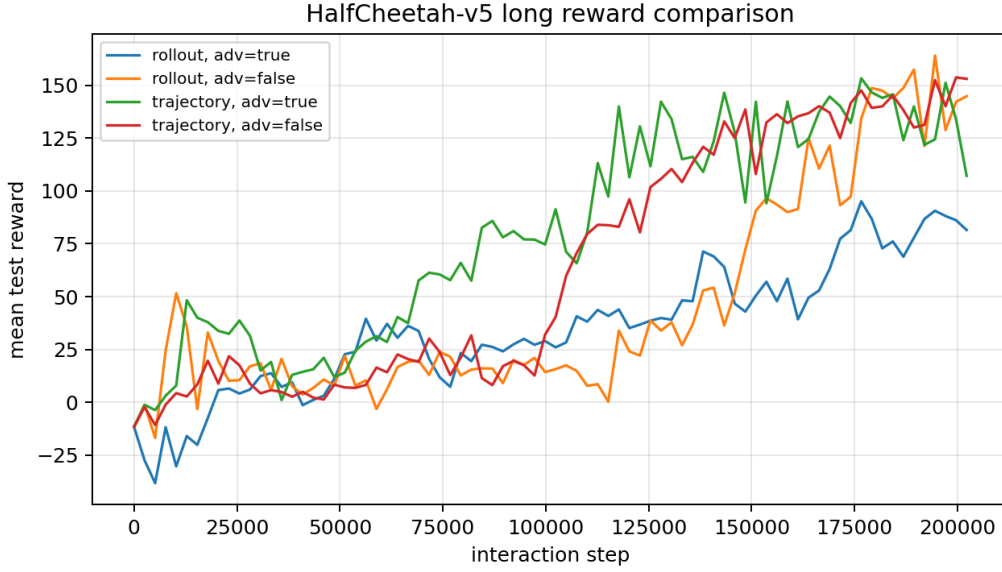


图 6: HalfCheetah-v5 长期训练 reward 曲线对比

HalfCheetah-v5 在长期训练后相比短期有明显提升, 其中 `trajectory + advan_norm=false` 最终 reward 约为 153.1, `rollout + advan_norm=false` 最终约为 144.9, 是该环境中表现较好的两组。`advan_norm=true` 的两组整体低于未归一化组。后期曲线仍有一定波动, 但已经出现较稳定的提升趋势, 可以认为接近阶段性收敛, 但还不是完全稳定收敛。

3.2.2 Hopper-v5

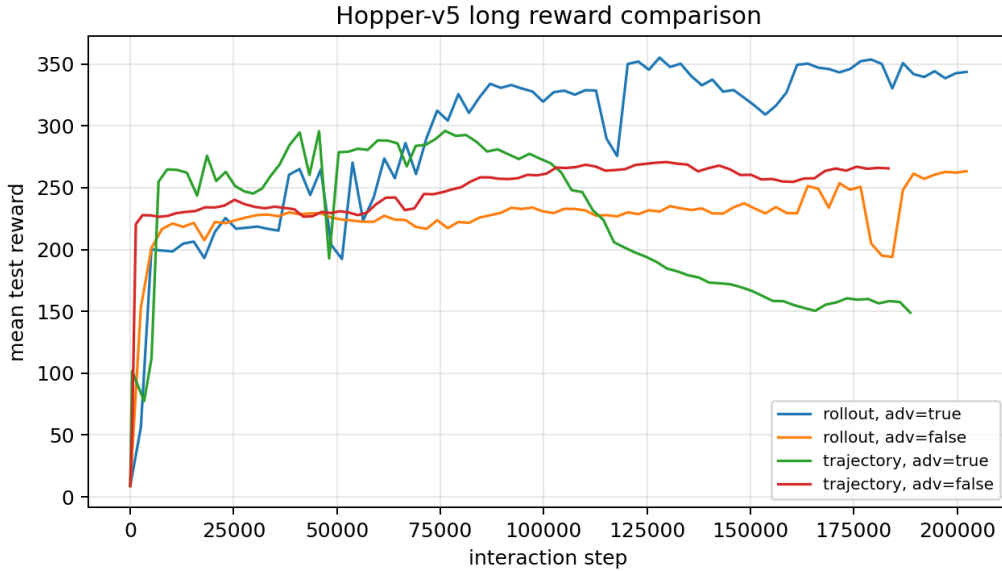


图 7: Hopper-v5 长期训练 reward 曲线对比

Hopper-v5 是长期实验中最接近收敛的环境。`rollout + advan_norm=true` 最终 reward 约为 343.6, 且最后若干次测试基本稳定在 340 左右; `rollout + advan_norm=false` 和 `trajectory + advan_norm=false` 也稳定在约 260 附近。相比之下, `trajectory + advan_norm=true` 后

期表现较差。总体而言，Hopper-v5 的 reward 曲线已经形成较明显的平台，基本达到阶段性收敛。

3.2.3 Walker2d-v5

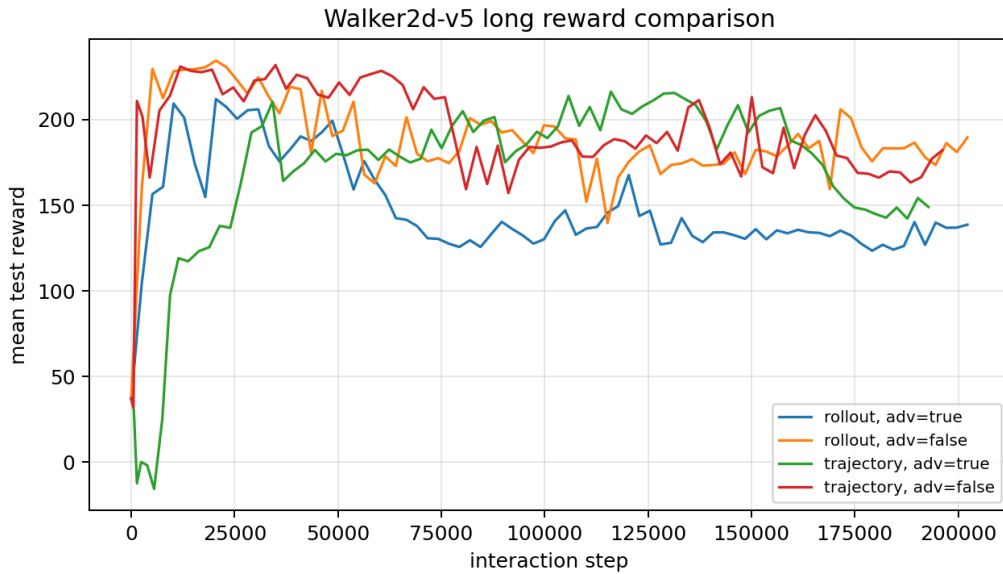


图 8: Walker2d-v5 长期训练 reward 曲线对比

Walker2d-v5 在长期训练中虽然相比初始阶段有明显提升，但后期震荡较大，尚未形成稳定平台。`rollout + advan_norm=false` 的最终 reward 约为 189.7，最好值约为 234.5；`trajectory + advan_norm=false` 最终约为 182.2，最好值约为 231.9。两组 `advan_norm=true` 的表现相对较弱。由此可见，Walker2d-v5 已经学到一定策略，但还没有达到稳定收敛，因此后续又对该环境进行了额外补充训练。

3.3 Walker2d-v5 额外训练结果

由于 Walker2d-v5 在长期实验中仍未形成稳定平台，因此在原有实验基础上继续进行了额外训练。图中给出了额外训练阶段四组设置的 reward 曲线。

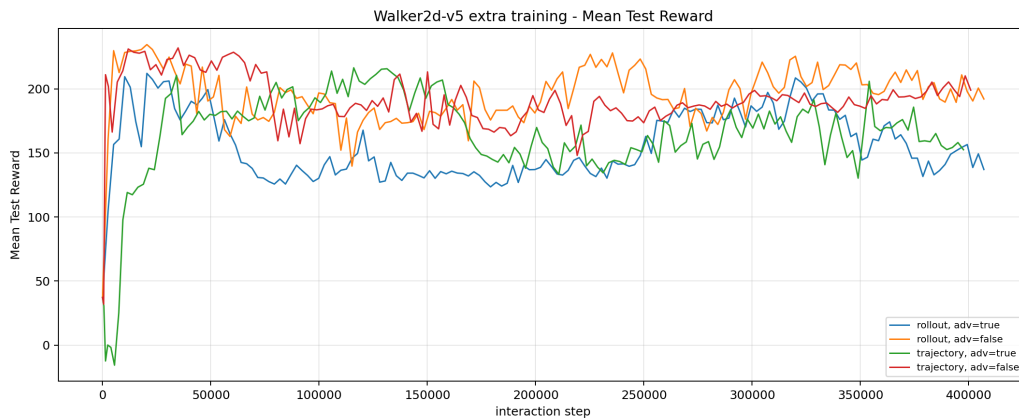


图 9: Walker2d-v5 额外训练 reward 曲线对比

从额外训练结果看，Walker2d-v5 的表现仍然具有明显波动。`trajectory + advan_norm=false`

的最终 reward 约为 198.9, 最好值约为 231.9; rollout + advan_norm=false 的最终 reward 约为 192.0, 最好值约为 234.5。相比之下, 两组 advan_norm=true 的最终表现较弱, 分别约为 137.0 和 152.4。说明在该环境中, 不进行 advantage normalization 的设置在本实验超参数下更容易获得较高 reward, 但额外训练并没有使曲线进入稳定收敛平台, 最好值和最终值之间仍存在较大差距。随后又尝试在已有 checkpoint 基础上继续训练, 并以是否收敛或达到时间限制 (8 小时) 作为停止条件。继续训练过程中的 reward 曲线如下。

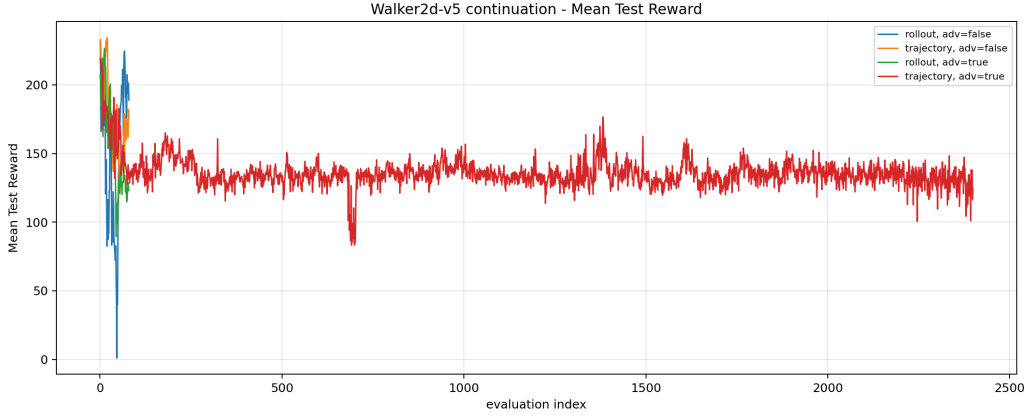


图 10: Walker2d-v5 继续训练至收敛尝试的 reward 曲线

继续训练后, rollout + advan_norm=false 的最好 reward 约为 231.5, 最终 reward 约为 188.9; trajectory + advan_norm=false 的最好 reward 约为 234.4, 最终 reward 约为 176.0。两组 advan_norm=true 的最好值虽然也超过 200, 但最终 reward 下降到 120 左右。由此可见, 继续训练并没有带来稳定收敛, 反而表现出较明显的策略退化和震荡。训练日志中还出现过 actor 输出为 NaN 的数值错误, 说明 Walker2d-v5 对当前 TRPO 实现和超参数更加敏感, 可能受到价值函数估计误差、重要性采样比值过大、KL 约束不够稳定以及缺少观测归一化或梯度裁剪等因素影响。因此, 本实验中 Walker2d-v5 只能认为策略有一定改进, 但尚未达到接近收敛状态。

3.4 Walker2d-v5 新参数补充训练结果

针对 Walker2d-v5 在统一超参数下难以收敛的问题, 本实验进一步为该环境单独设置了一组更保守的超参数, 并只保留表现较好的 rollout + advan_norm=false 设置进行补充训练。具体而言, KL 约束参数设置为 $\delta=0.003$, critic 学习率设置为 $3e-4$, critic 每轮更新步数设置为 15, 训练环境数和测试环境数均设置为 4, 最大 episode 长度设置为 500。第一小时训练结束后, 继续从该轮训练得到的 best_model.pth 出发进行第二小时续训。

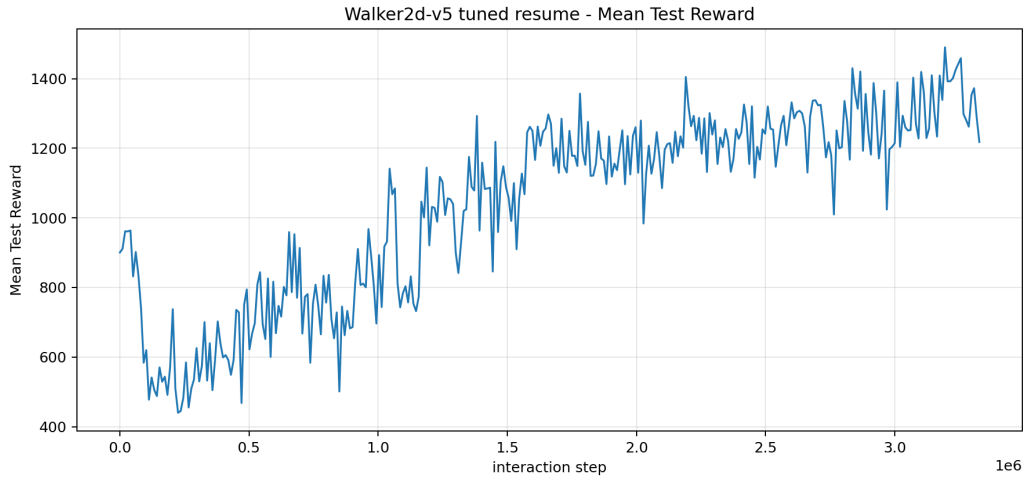


图 11: Walker2d-v5 新参数续训 reward 曲线

从 reward 曲线可以看出，新参数对 Walker2d-v5 的改善十分明显。原统一超参数下 Walker2d-v5 的最好 reward 约为 230 左右，最终 reward 约在 180–190 附近；而在新参数补充训练后，最好 reward 提升到约 1490.2，最终 reward 约为 1218.1，最后阶段平均 reward 约为 1316.4。相比原实验，新参数使策略从低回报震荡状态进入了较高回报区间，说明策略已经学到较稳定的行走行为。

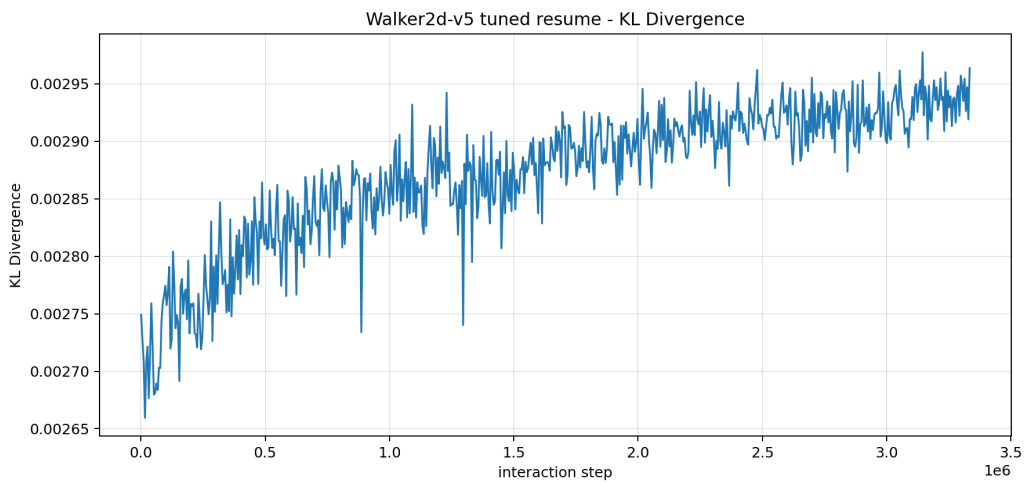


图 12: Walker2d-v5 新参数续训 KL 曲线

KL 曲线整体维持在较小范围内，最终 KL 约为 0.003，与设置的 `delta=0.003` 基本一致，说明策略更新幅度受到了较好的约束。相较于前面旧参数继续训练中出现的数值不稳定现象，新参数下没有再出现明显的 NaN 错误，训练过程更加稳定。

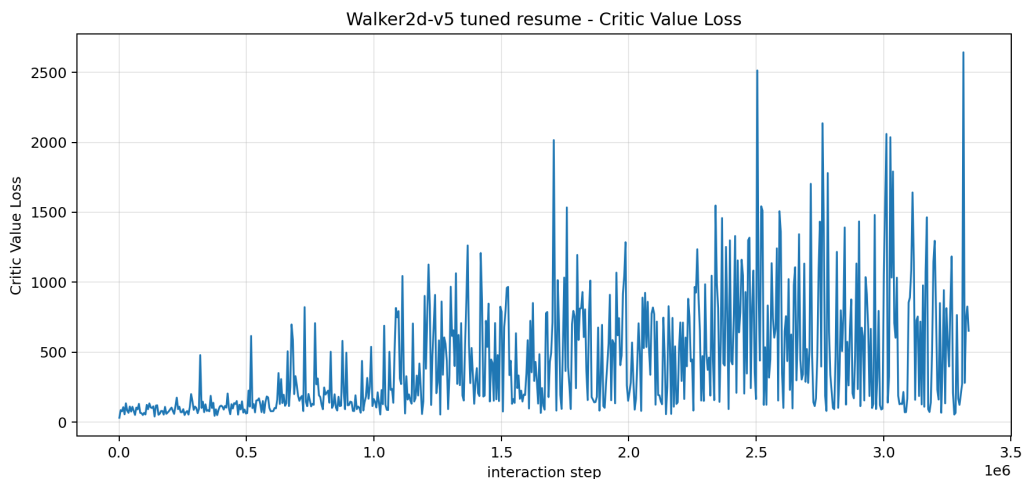


图 13: Walker2d-v5 新参数续训 value loss 曲线

value loss 在训练过程中仍存在一定波动，说明价值函数估计还没有完全稳定，这也是 reward 后期仍有震荡的重要原因之一。第二小时续训结束时，最后阶段 reward 波动范围约为 240.9，因此若按照非常严格的收敛判据，Walker2d-v5 仍不能认为已经完全收敛。但从实验目标和曲线表现来看，reward 已经长期保持在 1200 以上的较高水平，并且最好值达到 1490.2，相比原统一超参数结果有显著提升。因此，本实验认为 Walker2d-v5 在新参数补充训练后已经达到阶段性收敛，但尚未达到完全稳定收敛。

4 实验探索问题

1. 在相同超参数设置下，三个环境的训练难度是否一致？请结合 reward 曲线进行分析。

答：不一致。以长期训练的三个环境来看，Walker2d-v5 最难训练，训练结束后仍然有明显震荡，HalfCheetah-v5 其次，reward 曲线总体有收敛趋势但仍有些许波动，最简单的是 Hopper-v5，最终 reward 约为 343.6，最后阶段基本稳定在 340 附近，可以认为已经达到阶段性收敛。

2. 哪个环境中 trajectory 与 rollout 的样本利用率差异最明显？可能的原因是什么？

答：最明显的是 Walker2d-v5.rollout 和 trajectory 的样本利用率的差异主要在于因环境提前终止而导致 trajectory 中一条轨迹的有效样本减少。Walker2d-v5 中智能体动作更加不稳定，导致环境提前终止可能最大，因此样本利用率也差异最大。

3. 在 trajectory 模式和 rollout 模式下，advantage 与 return 的计算方式有什么不同？

答：两者都使用 GAE 计算 advantage，对于一条完整轨迹而言是一样的，主要差别在于数据边界处理。对于 trajectory 每一条轨迹都是完整的，但是存在大量人为填充的 0，因此需要 mask 变量来提取真实数据；对于 rollout 而言最后一条轨迹可能到一半就结束了，此时需要用 $V(s)$ 来估计未结束轨迹的后续价值。returns 的计算没什么区别，都是 $A+V$ 。

4.1 Advantage Normalization 的影响

1. 请分别设置 `advan_norm=true` 和 `advan_norm=false` 进行实验。两组实验的平均回报曲线有何差异？

答: 在 Hopper-v5 中, rollout + advan_norm=true 表现最好, 长期最终 reward 约为 343.6, 明显高于 rollout + advan_norm=false 的约 263.3。这说明在 Hopper-v5 的 rollout 模式下, advantage normalization 对稳定策略更新有明显帮助。但在 HalfCheetah-v5 和 Walker2d-v5 中, advan_norm=false 的配置反而更强。HalfCheetah-v5 中, trajectory + advan_norm=false 最终约为 153.1, 高于 trajectory + advan_norm=true 的约 107.2; Walker2d-v5 中, rollout + advan_norm=false 和 trajectory + advan_norm=false 也明显优于对应的 advan_norm=true 组。推测 advan_norm=false 在部分环境中保留了 advantage 的原始尺度, 反而可能带来更强更新。

2. 加入 advantage normalization 后, 训练过程是否更加稳定? 请结合 reward 曲线、KL divergence 或 surrogate objective 的变化进行分析。

答: advan_norm=true 通常让数值尺度更稳定, 但不一定得到更高 reward。从实验结果看, advan_norm=true 的 surrogate objective 通常更接近较小范围, 例如很多组最终 surrogate 约在 0.05 到 0.2 左右; 而 advan_norm=false 时, surrogate objective 可能明显更大, 例如 Walker2d-v5 或 Hopper-v5 中未归一化组出现了更大的 surrogate 数值。KL divergence 方面, 大多数组都被约束控制在较小范围内, 通常低于或接近 $\delta=0.01$ 。这说明 line search 和 KL 约束在发挥作用。advantage normalization 对 KL 的直接影响不如对 surrogate objective 明显, 但它会通过改变梯度尺度间接影响更新稳定性。

3. 在你选择的测试环境中, advantage normalization 是否总是带来性能提升? 如果不是, 可能的原因是什么?

答: 不会, 正如上面两问所言。原始 advantage 的尺度可能本身包含有用信息, 直接反映了动作好坏的强弱程度; 归一化后这些尺度信息被压缩, 可能导致更新幅度不够合适。normalization 只能保证稳定性而不能保证最优性。

4.2 Trajectory 与 Rollout 的样本利用率比较

1. 在 rollout 模式下, 每次策略更新实际使用了多少个 transition? 在 trajectory 模式下, 每次策略更新又实际使用了多少个有效 transition?

答: 本实验设置为: num_training_envs = 2, interact_per_epoch = 128, trajnum = 2, max_episode_length = 128。因此 rollout 模式每次策略更新固定使用 256 个 transition, trajectory 模式理论上最多使用 256 个 transition, 但实际取决于环境终止的情况。

2. 对比 trajectory 与 rollout 两种模式, 哪一种样本利用率更高?

答: 显然是 rollout。

3. 样本利用率更高是否一定意味着训练效果更好? 请结合实验曲线进行分析。

答: 不一定。在 HalfCheetah-v5 的长期实验中, trajectory + advan_norm=false 的最终 reward 约为 153.1, 略高于 rollout + advan_norm=false 的约 144.9。这说明即使 trajectory 的样本利用率可能更低, 它仍可能因为使用完整轨迹而获得更合适的 return 和 advantage 估计。而在 Hopper-v5 中, rollout + advan_norm=true 表现最好, 最终 reward 约为 343.6, 说明 rollout 的稳定样本量确实带来了较好的训练效果。原因可能是更多的样本可能可以保证训练的稳定性, 但是想要获得更好的效果, 可能还受环境、轨迹终止频率、advantage 估计准确度、critic 拟合度等众多因素影响。使用 trajectory 可能不能获得更多的样本, 但它的每一条轨迹都是自然的, 可能获得更好的 advantage 估计。

5 总结与反思

1. 在本项目中，`collect_trajectories` 的实现是否会改变 TRPO 的 on-policy 特性？说出你的判断理由。

答：没有改变。on-policy 即需满足更新所用数据均为当前策略采出。而 `collect_trajectories` 每轮开始时都会重置环境且清空原有数据，确保更新所用数据都是当前 agent 交互得到的数据。

2. 如果只能保留一种采样方式，你会选择 trajectory 还是 rollout？请结合样本利用率和训练性能说明理由。

答：我选择 rollout。首先它的样本利用率会高一些，起码能够保证稳定性。Hopper-v5 中 `rollout + advan_norm=true` 是长期实验中表现最好的配置，最终 reward 约为 343.6，并且后期较稳定。可能某些环境 trajectory 更适合，但 rollout 更有“中庸之道”的感觉，非常稳定，如果它不行再去试别的方法。

3. 完成实验报告的撰写后，你对 TRPO 算法是否有了新的感悟或认识？（这个问题不建议 AI 辅助生成，AI 时代最珍贵的是人类自我的想法。相信你写完报告后总会有一些自己的思考，或许是疑惑或许是感悟，完善与否不重要，写出你自己的想法就好。）

答：TRPO 的理论推导的确非常漂亮，而且用到了非常多的数学知识。然而在实际工程实现方面存在诸多难以解决的问题，如一开始的 TV 散度不可微，海森矩阵计算占比太大等等。虽然算法上有诸多优化和近似的技巧，比如改为使用 KL 散度，泰勒展开近似，共轭梯度求解向量等等，并且还近似后的结果做出安全检测（线性搜索）。但是得到的算法还是非常复杂，运算成本大，TRPO 对环境和超参数的敏感性很强。最让我印象深刻的是 walker 2d-v5 环境，在经过短期、长期、额外训练后依然得到的结果差强人意，并且浪费了大量的时间和算力，最后在 ai 辅助下重新设计了参数针对性地对该环境做实验，才得到了比较好的环境。而这个环境并不是最复杂的，本次选取的三个长期训练环境都是稍简单的，如果长期训练最复杂的 Humanoid-v5 环境，可能在参数调试和优化技巧上会耗费更多的时间精力。在实际工程中，面对复杂度远超出此的环境，TRPO 可能需要耗费巨量的时间算力，而各种超参数设计、采样方式、优化方式的选取，有一种“炼丹”的感觉。可能正是这种困难使得在 TRPO 的基础上进一步提出了 PPO 的方法。

4. 针对本次算法实现，你是否有什么想法和想改进的建议，你在实现过程中有没有因为项目 README.md 或者实验报告引导和介绍问题导致卡了很久，如果有，请详细说明，并给出你的建议。关于代码/实验报告问题难度你是否有感觉很难/简单？

答：本次实验报告和介绍问题引导我去更多地思考这个算法，还是起到了不错的正向效果。单纯自己学一遍的话印象没有这么深。在实验过程中唯一被卡住的就是 walker 环境一直效果很差，不过在 ai 的帮助下还是解决了。对于报告问题来说并不算很难，我对于这个算法本身和代码设计思路基础还是比较清晰的。困难点更多在于，这个代码项目本身比较大，且使用了很多工程/项目化的设计方法，跟我平时接触到的代码不太一样，读和写还是非常吃力，需要严重依赖 ai。